

Building an AI-Powered, Prompt-Driven Floor Plan Platform

A feasibility and technical-build assessment

1. What you're actually asking to build

The screenshot shows more than “a floor plan.” It contains four distinct deliverables stacked together:

1. **Two dimensioned 2D plans** (ground floor + first floor) with structural walls, room labels, and stairs.
2. **A door and window schedule** (D1-D4, W1-W4) with real millimetre dimensions.
3. **Photorealistic 3D exterior renders** of the finished house.
4. **Metric accuracy** — the plan is drawn to scale and could, in principle, be built from.

That matters because “generate a floor plan from a prompt” and “generate *that whole package* from a prompt, then let me re-tweak it by editing the prompt” are two very different engineering problems. The first is largely solved and commoditised. The second — especially the *edit-by-re-prompting* part, at metric accuracy, for **detached Nigerian houses** — sits right at the current research frontier. This report separates what you can buy today from what you'd have to build.

2. The state of play in 2026 (what already exists)

Prompt-to-floor-plan is no longer research-only; it's a live product category.

Commercial tools shipping today:

- **Maket.ai** — the clearest reference point. Montreal-based, founded ~2020, publicly launched 2023, over 1 million registered users on a team of roughly 15 people, and about US\$4M CAD raised in total (a \$3.4M CAD seed led by Amiral Ventures in late 2025). It takes room types, quantities, target square footage and lot shape — via dropdowns or natural language — and produces multiple layouts in seconds, plus text-based optimisation, 3D renders, zoning-compliance checks, and DXF/PDF/PNG export. Pricing anchors the market: free tier (50 one-time credits, single storey), Pro at ~US\$20/month (300 credits, multi-storey up to 4 floors, text-based editing). They openly call it “ChatGPT for architecture.”
- **TestFit** — multi-family and site-feasibility focused (developer economics, not single homes).
- **Finch3D** — early-stage layout optimisation.
- **Planner 5D, RoomGPT, Homestyler, FloorPlan AI, Drafted** — consumer/prosumer tools; several now do text-to-plan and even export GLB 3D models.

What this tells you: the *category* is proven, a well-funded team got to 1M users with ~15 people, and unit economics run on a credits model. But note what every honest review says — **these tools are pre-design and feasibility aids. Their output still needs an architect's eye before it becomes buildable construction documents.** None of them reliably produces a code-compliant, structurally-sound, fully-dimensioned drawing set for a specific plot. That gap is exactly where a trust-focused platform can differentiate.

3. How the “engine” actually works

There is no single technique. There are five families, and a serious product usually combines them. Understanding the trade-offs is the core of the build decision.

A. Raster / image models (GANs). Early approach (HouseGAN, HouseGAN++). The plan is treated as an image and generated pixel-by-pixel. Fast and visually plausible, but the output is *not metric* (no reliable dimensions), rooms can duplicate or go missing, and it ignores hard constraints. Not suitable on its own for buildable output.

B. Graph-conditioned generation (the “bubble diagram”). Almost every good system represents a house first as a graph: nodes = rooms (bedroom, kitchen, bath), edges = “these two connect / are adjacent.” The model then realises

that graph into geometry. This is what lets a user say “put the kitchen next to the dining, master bath en-suite” and have it respected. Graph neural networks and graph-transformers do this.

C. Vector diffusion (HouseDiffusion and successors). Instead of pixels, the model denoises the actual *coordinates* of room polygons. This is the current quality leader for geometry — it produces clean vector plans (walls as real lines, exportable to CAD), handles non-rectangular shapes, and respects the input graph far better than GANs. This is closest to producing something CAD-ready.

D. LLM + structured representation (the fast-moving frontier). The newest and, for your goals, most important line. Instead of generating geometry directly, a large language model emits a **compact structured description** of the house (a JSON layout, a hypergraph, or a partition tree), which is then rendered into geometry. Recent work (HypergraphFormer, from an Autodesk collaboration; DStruct2Design; and LLM-with-reinforcement-learning approaches fine-tuning models like Llama-3.3-70B) found something crucial:

The thing that unlocks editability is the representation, not the model size. When the house is a small structured object, a few-token change to that object equals a meaningful structural edit — with no full re-generation. When the house is pixels or raw polygons, every edit is a fresh guess.

This is the direct answer to your “tweak it by changing the prompt” requirement.

E. Prompt-guided generation and editing (ChatHouseDiffusion). The most on-the-nose piece of prior art for what you described. It uses an LLM to parse natural language, a graph encoder for room relationships, and a diffusion model to generate — and critically, by swapping the model’s attention maps, it does **precise local edits without redesigning the whole plan**. So “make bedroom 3 bigger and move the balcony to the east” changes only what you asked, not the entire house. That “surgical edit, not full redraw” behaviour is the single feature that will make or break the user experience you’re after.

4. The full pipeline your product would need

To go from a sentence to the *package* in that screenshot:

1. **Prompt understanding.** LLM parses “4-bedroom two-storey house, 8.4m × 12m plot, ensuite master, parking for two cars, balcony over the entrance” into a structured spec + a room-adjacency graph.
 2. **Constraint layer.** Inject hard rules: plot dimensions, setbacks, minimum room sizes, corridor widths, stair geometry, structural grid.
 3. **Layout generation.** A diffusion or LLM-structured model produces the room polygons per floor, respecting the graph and constraints.
 4. **Vectorisation + dimensioning.** Convert to CAD linework, auto-place dimension strings, generate the door/window schedule.
 5. **Multi-floor coordination.** Ensure the first floor stacks correctly on the ground floor (load-bearing walls align, stairs land in the right place). This is genuinely hard and where most tools cheat.
 6. **3D + render.** Extrude the 2D plan into a 3D model (GLB/IFC), apply materials, and render the exterior — either by 3D engine or by an image model (ControlNet-style) conditioned on the massing.
 7. **Edit loop.** Store the structured spec so a *new prompt* mutates the existing spec and re-renders only what changed — not a fresh house each time.
 8. **Export.** DXF/DWG for CAD, IFC for BIM, PDF for clients.
-

5. The five hard problems (be honest about these)

Problem 1 — Data, and this is the big one for Nigeria. Nearly every model in the literature is trained on **RPLAN**: ~80,000 Asian *apartments*, 60–120 m², densely packed, non-commercial-licence-only. Other datasets (LIFULL — Japanese; Swiss Dwellings; MagicPlan; CubiCasa5K) are Western or East Asian. The documented finding is brutal: models suffer **up to an order-of-magnitude performance collapse when moved to a different region’s building conventions**, because layouts vary by architectural culture, climate, plot shape and construction practice. A Nigerian detached duplex — external staircase, boys’ quarters, generator house, larger plots, cross-ventilation, security-driven

layout — is **out of distribution** for every off-the-shelf model. **You would have to build a Nigerian/West-African floor plan dataset.** This is your hardest, most expensive, and most defensible asset. It's also perfectly aligned with what Genuine Stuff already is: a trust-and-verification network with supplier and project relationships that generate real drawings.

Problem 2 — Metric accuracy and buildability. Pretty pictures are easy; *correct* is hard. Walls that actually align floor-to-floor, stairs that fit the storey height, doors that open, dimensions that add up to the plot — image models routinely fail here. This is why vector/structured approaches matter and why a human architect sign-off stays in the loop for anything that gets built.

Problem 3 — The edit experience. Making “change the prompt to tweak the plan” feel *surgical* rather than *slot-machine* (where each edit gives a random new house) is a real UX and modelling challenge. The answer is the structured-representation approach in §3D/E — keep a persistent spec, mutate it, re-render deltas.

Problem 4 — Code compliance. This is where trust lives, and it's very location-specific: Nigerian National Building Code, state planning setbacks, plot coverage ratios, minimum room and window sizes, fire egress. Newer LLM-based compliance engines can interpret rules *contextually* (not just numeric thresholds) and flag the specific element that fails with remediation guidance — a strong differentiator, but you'd need to encode Nigerian rules yourself, as no dataset ships them.

Problem 5 — 2D → photorealistic 3D. The renders in your screenshot are the “wow.” Getting there reliably (consistent with the plan, not a hallucinated different house) is a separate model and a common failure point.

6. Three ways to build it — pick a lane

Lane 1 — Wrap and orchestrate (weeks, lowest cost, lowest moat). Sit an interface and workflow on top of existing APIs (a Maket-style generator + a render model + your own material/supplier layer). You ship fast, you learn what Nigerian users actually ask for, and you start collecting prompts and edits as data. Weakness: output quality inherits the domain-shift problem, and your moat is thin until you add proprietary data. *This is almost certainly your correct starting point.*

Lane 2 — Fine-tune open models on curated data (months, medium cost, real moat). Take open architectures (a HouseDiffusion-style vector model, or an LLM-structured generator) and fine-tune on a Nigerian dataset you assemble. Techniques like LoRA make this affordable — recent work fine-tuned a 70B model on a handful of GPUs. This is where you'd actually solve the domain-shift problem and produce plans that look and build like Nigerian houses.

Lane 3 — Train bespoke models + compliance engine + BIM export (year+, high cost, deep moat). Full “construction operating system” territory: your own generator, Nigerian-code compliance layer, material takeoff, and supplier routing. This is the long-term vision, not the first release.

7. What it takes — resources

- **Team (minimum serious build, Lane 2):** 1 ML engineer (generative/diffusion + LLM fine-tuning), 1 full-stack engineer, 1 architect/BIM specialist (the domain expert who defines “correct”), 1 product/design. A part-time quantity surveyor once you connect to material takeoff.
- **Data:** the real work. Sourcing, cleaning, and vectorising thousands of Nigerian residential drawings (from architects, developers, your own supplier/project network), annotated with room types, adjacencies, dimensions, and ideally material specs. Budget for this as your primary line item — it *is* the product.
- **Compute:** modest for wrapping (API costs only); meaningful GPU time for fine-tuning (rentable H100 clusters, not owned hardware); ongoing inference cost per generation drives your credits pricing.
- **Money & time, rough order of magnitude:** Lane 1 MVP in a quarter on a lean budget; Lane 2 a credible year to a differentiated product. For reference, a focused team of ~15 reached 1M users and a real product on ~US\$4M total funding — so this is fundable at seed scale, but the *data* is what separates a wrapper from a moat.

8. Why this is unusually strong for Genuine Stuff specifically

Most floor-plan startups have a hard time monetising a free-ish design tool. You have the opposite problem solved before you start: **the floor plan is the top of your funnel, not the product.**

A generated, dimensioned plan gives you a **material takeoff** — quantities of blocks, cement, rebar, roofing, doors, windows, tiles. That takeoff routes straight into your verified-supplier marketplace with transparent pricing. The design tool becomes the on-ramp to procurement, and procurement is where the trust, verification and accountability you're building actually get paid for. Maket has publicly said material takeoffs and zoning verification are their *roadmap*; for you they're the *core business*. That's a genuine structural advantage, not marketing.

The honest caveat: this only works if the output is trustworthy. A plan that under-counts rebar or ignores a setback would damage the exact trust you're building. So the sequencing below is deliberately conservative.

9. Recommended path

1. **Start in Lane 1.** Ship a prompt-to-plan + render experience by orchestrating existing models. Goal is not perfect plans — it's real Nigerian users, real prompts, and real edit requests, captured as data.
2. **Attach a material takeoff to every plan** and wire it to your supplier network from day one. This is your differentiator and your revenue, and it's the part no floor-plan competitor is built to do.
3. **Build the Nigerian dataset in parallel.** Use your supplier and project relationships to source real drawings. This is the moat; treat it as the main investment.
4. **Move to Lane 2** once you have data and demand signal: fine-tune for Nigerian typologies and add the persistent-spec edit loop so re-prompting tweaks surgically.
5. **Keep an architect in the loop** for anything that gets built, and encode Nigerian code compliance as a checking layer before you ever market plans as buildable. This protects the trust that is the whole point of the company.

The short version: the technology to generate and re-prompt-edit floor plans exists and is improving monthly. The defensible product is not the generator — it's **Nigerian-specific data + compliance + the material-and-supplier layer underneath it**. That's the part only Genuine Stuff is positioned to build.

Sources & further reading

- Maket.ai — funding, team, product: BetaKit; TechFundingNews; maket.ai/pricing
- RPLAN dataset and domain-shift findings: “Mitigating Domain Shift in Conditioned Floor Plan Generation” (arXiv 2607.06483); “Space Syntax-guided Post-training” (arXiv 2602.22507)
- Core generative methods: HouseGAN++/HouseDiffusion (CVPR); “Generative Floor Plan Design with LLMs via RLVR” (arXiv 2605.14117)
- Prompt-guided editing: **ChatHouseDiffusion** (arXiv 2410.11908); **HypergraphFormer** (arXiv 2605.18932); DStruct2Design (arXiv 2407.15723); Tell2Design (ACL 2023)
- Compliance: “Towards an automated AI-based framework for floor plan compliance checks” (arXiv 2607.00015)
- Commercial landscape review: ArchPulse “Best AI Floor Plan Generators 2026”

Note: arXiv IDs are as returned in current search results and should be confirmed before citing formally.